



AULA 3

Data Lakes, Aquisição/Ingestão de dados e ETL (Extração, Transformação e Carga)



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Descrever ambientes de *data lake* e contrastá-los com ambientes de *data warehouse* (DW) “puro”.
- Descrever arquitetura de *business intelligence* (BI) centrada em *data lake* (DL).
- Conhecer o paradigma MapReduce e o framework Apache Spark
- Conhecer as principais estratégias para ingestão de dados para *data lakes*.
- Identificar as principais ferramentas de apoio à exploração de dados do ecossistema Hadoop Spark.
- Praticar ingestão de dados, uso de MapReduce, criação de tabelas no Hive, consultas SQL e *jobs* Spark.

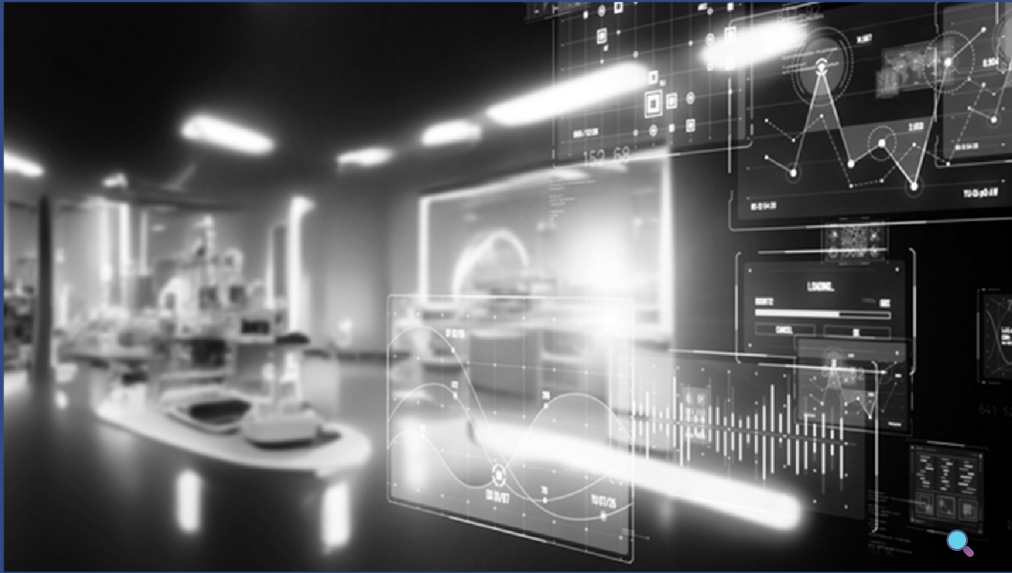
Que história é essa?

Nesta terceira e última aula da disciplina, definiremos o conceito de *data lake* e conheceremos as principais características de ambientes que o têm como elemento central (estendendo os ambientes centrados puramente em DW), bem como os principais componentes de uma infraestrutura de dados para BI centrada em *data lake*. Também conheceremos o papel importante que o paradigma MapReduce e componentes como Hadoop e Spark exercem em tais ambientes. Por fim, falaremos sobre como se dá a aquisição de



Nem tudo é tão “arrumadinho” assim: o surgimento (e inevitável crescimento) dos *data lakes* na era *big data*

Já vimos que a tão conhecida “era big data” é caracterizada por uma profusão de dados em grande volume, variedade e velocidade (intratável através de paradigmas de processamento de dados tradicionais). Outros Vs também complementam essa caraterização, embora não sejam essenciais ou consensualmente adotados na literatura.



Em conjunto com a maturidade tecnológica e com o avanço das infraestruturas de dados, essa profusão de dados tornou não apenas possível, mas também bastante atraente (e, com o passar do tempo, inevitável), que dados das mais diversas origens, independentemente de seu formato ou estrutura, fossem “objetos de desejo” dos ambientes de BI e de *data science*, portanto demandando que fossem integrados às fontes de dados internas (corporativas e setoriais) das organizações. Nesse contexto, surgiram os *data lakes*.

Data lakes são repositórios distribuídos que reúnem dados em qualquer escala, provenientes da própria organização ou de fontes externas, e que fazem parte da infraestrutura de armazenamento e processamento de *big data*.

Em um *data lake*, os dados são armazenados em seu formato nativo, ou seja, exatamente como foram gerados no dia a dia da organização (seja uma tabela em um banco de dados relacional, um arquivo XML, uma página *web* em HTML, uma planilha CSV, um *stream* de dados de um sensor ou mesmo arquivos em diferentes mídias, como documentos, imagens e vídeos).

Ter uma plataforma que seja capaz de ingerir dados diretamente dessas diversas origens, em diferentes formatos, é bastante desejável. Permitir que usuários da organização possam realizar consultas sobre esses dados, combinados com dados dos sistemas corporativos internos, é adequado à evolução de processos e procedimentos, há pouco tempo vinculados somente a dados produzidos pela própria organização.



Fique Ligado

A capacidade de ingerir todos esses dados (internos e externos) em uma plataforma única de armazenamento e processamento de dados permite às organizações evoluir no sentido da maior integração desses dados produzidos pela organização com dados produzidos por outras organizações, acrescentando valor substancial às análises que embasam as decisões a serem tomadas.

A ideia central por trás dos *data lakes* é que o fato de se ter o dado na forma como ele foi originalmente criado e coletado no ambiente organizacional abre caminho para uma série de possibilidades de uso, e de reúso, do portfólio de informações de uma organização, com diversas possibilidades de tratamento dos dados, definidas a cada uso.



De olho no mercado

Isso inclui, por exemplo, a construção de painéis e *dashboards*, execução de algoritmos de aprendizado de máquina para construção de modelos, processamento *big data* em tempo real, entre outras aplicações.

Da mesma forma que um DW, pode-se se entender que um *data lake* é uma abordagem de integração dos dados de uma organização, uma vez que nessa arquitetura os dados (identificados previamente como potencialmente relevantes) são reunidos e tornados acessíveis aos usuários através de uma mesma plataforma. No entanto, essas duas abordagens apresentam características bem distintas e complementares.

Data lakes e data warehouses

Os DL diferem dos tradicionais DW em vários aspectos. Confira:

Interativo

Interativo que contem 6 imagens

Em ambientes de BI modernos, é frequente que os DWs estejam incorporados à arquitetura multiplataforma de um DL como um de seus componentes de armazenamento de dados estruturados e seguindo o paradigma dimensional.



De olho no mercado

Ainda, uma estratégia comum que as organizações adotam para modernizar o seu ambiente de BI a partir do DW e de *data marts* associados é incorporar processamento massivamente paralelo (*massive parallel processing* – MPP) sobre uma arquitetura distribuída ou *shared-nothing* (na qual cada nó tem seu próprio disco, memória e processador locais) com nós especializados para armazenamento e processamento, suporte ao particionamento de dados e sistemas de armazenamento de dados semi (ou não) estruturados, como sistemas de gerenciamento de bancos de dados (SGBDs) NoSQL, sobre uma arquitetura Hadoop.

Data lakes como infraestrutura de dados nas organizações

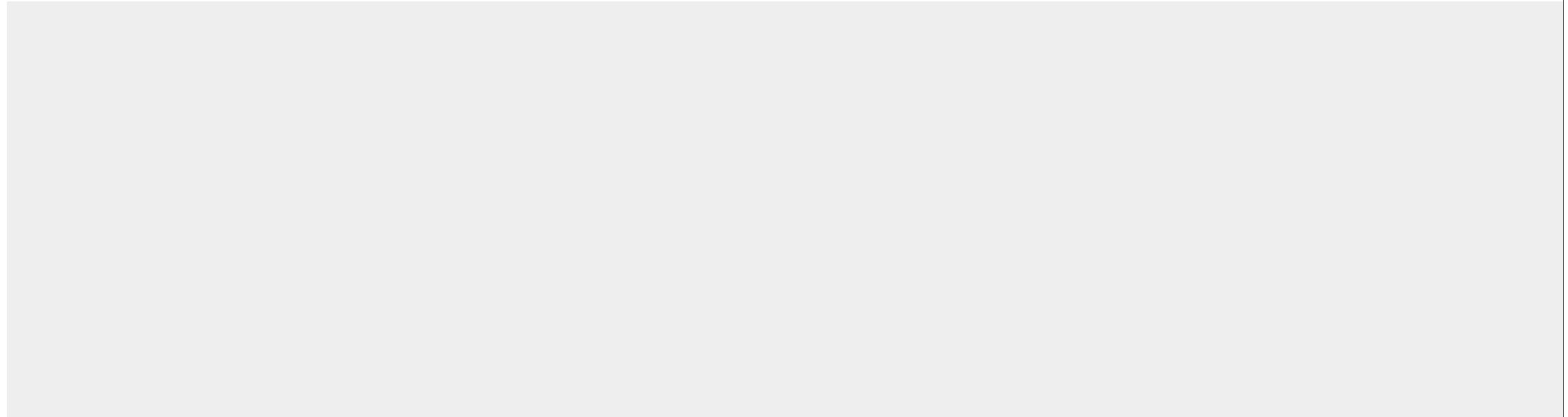
Como não existe uma definição consolidada e universalmente aceita para o que é de fato um DL, as funcionalidades de um ambiente de DL variam.

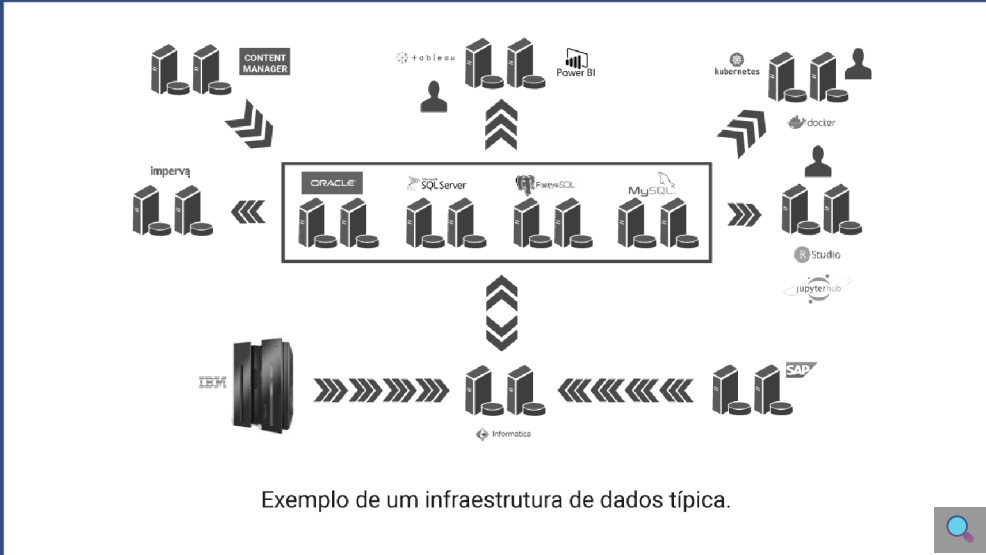


Alguns autores o definem “apenas” como um **repositório de dados** (GRÖGER, 2018; TAHER *et al.*, 2016; KATHIRAVELU; SHARMA, 2017); já outros o caracterizam como um **ecossistema completo**, ou seja, uma *infraestrutura de dados* que engloba desde componentes para a aquisição de dados até componentes para suporte à visualização da informação (MCPADDEN *et al.*, 2018; BHANDARKAR, 2017; NOGUEIRA; ROMDHANE; DARMONT, 2018).

Uma infraestrutura de dados é composta tipicamente por diversos componentes que colaboram para armazenar e processar dados. Sistemas gerenciadores de bancos de dados, plataformas de ETL e ferramentas de visualização de dados estão entre os principais componentes de uma infraestrutura de dados.

Veja um exemplo da infraestrutura de dados de um DL na figura a seguir.



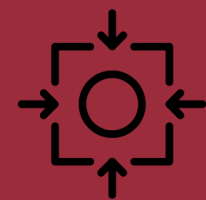


No centro da figura estão os bancos de dados corporativos, onde estão depositados dados de sistemas transacionais e DW, servidos frequentemente por *clusters* de servidores, que oferecem alta disponibilidade e desempenho adequado para receber e processar solicitações de atualização e consultas a esses dados. Outros repositórios de dados incluem arquivos e bancos de dados em *mainframes* e em sistemas *enterprise resource planning* (ERP), que servem a toda a organização.



Dados de diferentes origens trafegam em plataformas ETL para serem acomodados em bancos de dados corporativos, atraídos pelas aplicações que fazem uso desses dados. Assim, é comum que em uma infraestrutura de dados de uma grande organização existam centenas ou milhares de *workflows* ETL transferindo dados entre SGBDs, do *mainframe* para um banco de dados ou do ERP para um banco de dados.

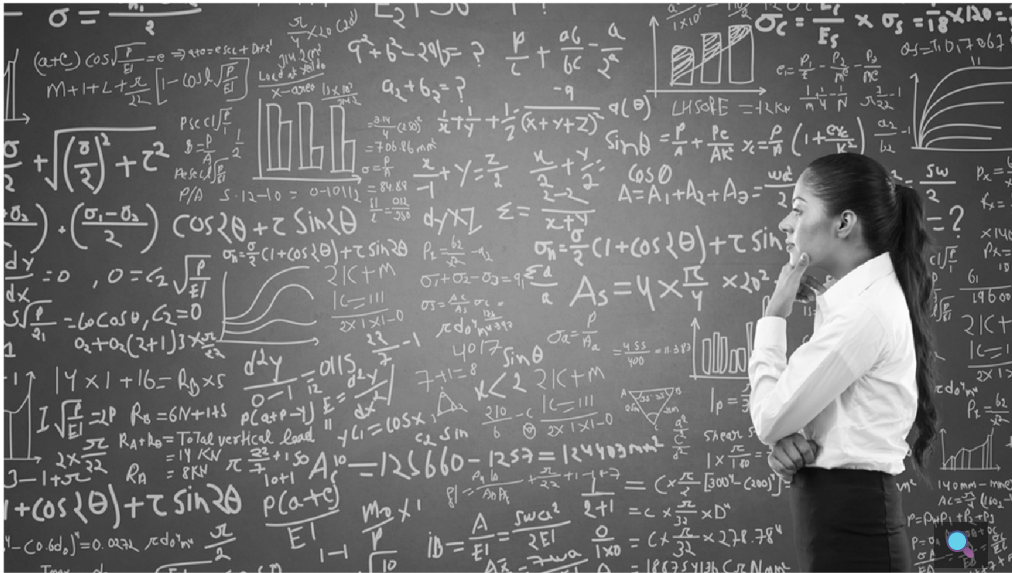
Esses *workflows* ETL fazem parte do conjunto de tarefas computacionais executados durante a Produção Batch de uma organização, que consiste de todas as tarefas de processamento em lote executadas entre o fim de um expediente e o início do próximo, responsáveis por preparar dados para diversos sistemas, permitindo que o dia útil seja iniciado com informações atualizadas para todos os usuários da organização. Garantir que os procedimentos da Produção Batch sejam executados dentro da janela disponível é muitas vezes um desafio a ser superado pelas equipes técnicas de infraestrutura de dados.



Mantendo o foco

Como advento da era *big data*, como já vimos, os requisitos das infraestruturas de dados se ampliaram enormemente, demandando maior poder de processamento e escalabilidade, tratamento de diversos formatos de dados e customização e personalização das análises para cada perfil de usuário e aplicação.

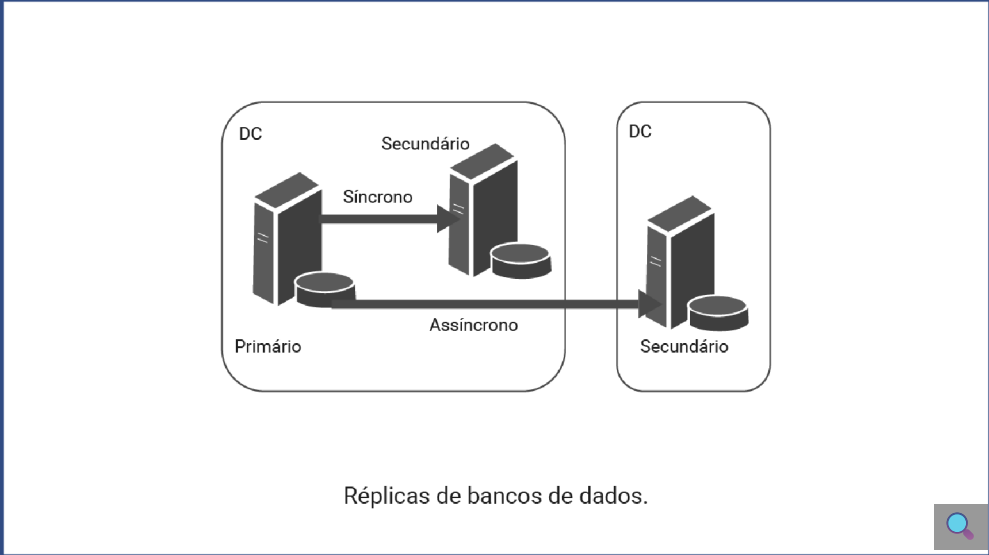
Nesse contexto, exemplos de ferramentas frequentemente incorporadas à infraestrutura de dados de um DL foram R Studio e JupyterHub (ambientes de programação com acesso direto aos bancos de dados) e plataformas de containerização (como Docker e Kubernetes), que permitiram maior agilidade na implantação de soluções não só de dados, mas também de aplicações.



Ainda, questões que envolvem redundância no armazenamento de dados, computação paralela e distribuída e escalabilidade, tanto na ingestão quanto no processamento dos dados, há muito são objeto de pesquisa e desenvolvimento. Especialmente quando o volume dos dados atinge terabytes, observa-se variedade nos formatos dos dados e aumento da velocidade de ingestão desses dados, características principais do que chamamos *big data*, torna-se fundamental a disponibilidade de uma plataforma que implemente as características citadas.

Dessa forma, estratégias de replicação de dados foram adotadas ao longo do tempo, não apenas visando ao aumento de desempenho através de paralelismo (quando o processamento de uma consulta pode ser dividido entre diversos nós trabalhando em paralelo), mas também para reduzir o impacto do acesso aos dados no ambiente de produção, ou mesmo por questões de segurança e contingência em caso de falhas.

Na figura a seguir, vemos uma estratégia clássica de replicação de bancos de dados, com uma réplica síncrona no *datacenter* principal e uma réplica assíncrona no *datacenter* de contingência. O acesso aos dados pode ser concedido em uma dessas réplicas, eliminando qualquer impacto que um acesso às bases de dados de produção possa causar, ainda que seja um acesso para somente leitura.



Por conta da variabilidade de funcionalidades que os DL podem oferecer, a sua arquitetura também pode apresentar diferentes configurações. No entanto, a maioria dos autores tende a concordar que o Hadoop é arquitetura mais comum para DL, apresentando-se tanto de forma *stand alone* quanto em combinação com outros componentes, como Spark e SGBDs NoSQL (COUTO, 2022).



GFS, MapReduce e Hadoop

Quando os mecanismos de busca enfrentaram dificuldades para o processamento de um volume cada vez maior de dados no início da década de 2000, pesquisadores se debruçaram sobre o problema. À época, Google, Yahoo!, Altavista, entre outros, atingiam o limite das ferramentas disponíveis para busca e indexação de conteúdo. Os *crawlers* disponíveis à época não se comportavam bem com o número crescente de páginas a serem atualizadas.

Nasce o projeto Nutch na Apache em 2002, pioneira do *software open source*, cujo *crawler* funcionava bem para certo conjunto de páginas e começou a apresentar problemas de desempenho com a explosão de páginas na *web* e a necessidade de processar milhões de páginas dentro da infraestrutura da época.

Esse aumento exponencial das páginas disponibilizadas na *web* colocou forte pressão também sobre as plataformas de armazenamento e processamento, o que os levou a pesquisar e desenvolver novas alternativas.

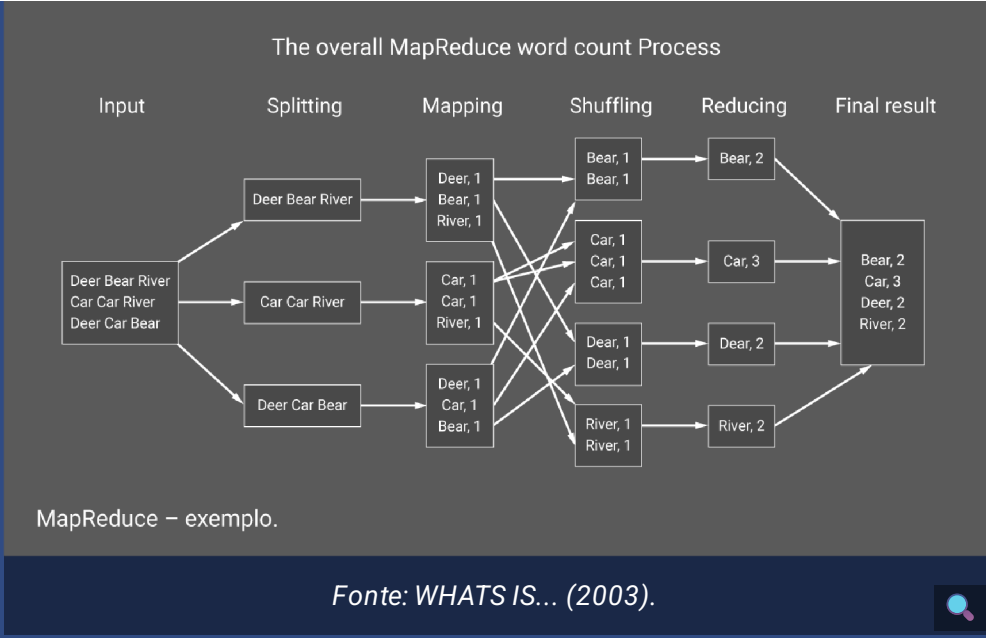


Estudo complementar

Naquela época, a Google se destacou por revolucionar completamente os modelos existentes de armazenamento e programação distribuída com as propostas do Google File System (GFS) (GHEMAWAT; GOBIOFF; LEUNG, 2003) para armazenamento redundante e tolerante a falhas e do MapReduce (DEAN; GHEMAWAT, 2004) como novo modelo de programação (e sua implementação) para execução de programas em um grande *cluster* de servidores de forma paralela e distribuída.

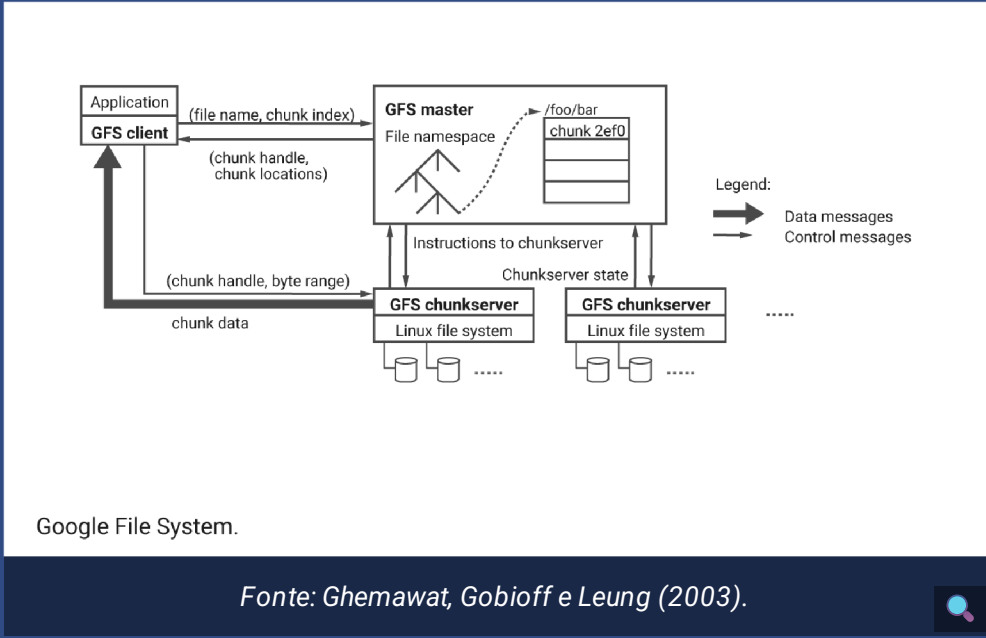
[Clique aqui](#) para ler mais sobre Google File System (em inglês)
[Clique aqui](#) para ler mais sobre MapReduce (em inglês)

A figura a seguir ilustra um exemplo de uso do MapReduce para retornar a contagem de palavras em um conjunto de dados. A entrada de dados (três linhas) é dividida em três partes na fase *splitting*, sendo cada uma encaminhada para uma tarefa do tipo *map*. Na fase *mapping*, cada tarefa *map* executa em um nó do *cluster* e opera somente sobre a sua entrada, gerando uma estrutura chave-valor. Em seguida, a fase *shuffling* transfere os dados dos nós *mappers* para os nós *reducers*. Antes da transferência, porém, todas as chaves são ordenadas de forma que as mesmas chaves sejam transferidas para o mesmo *reducer*. A fase *reduce* executa a agregação. No exemplo, essa agregação é o somatório dos valores contidos em cada chave, resultando no número de ocorrências de cada palavra da entrada original. Os *outputs* de cada *reducer* são, então, combinados para gerar o resultado final.



Já o GFS trata do problema do armazenamento distribuído, ou seja, um sistema de arquivos que não existe em uma única máquina, mas em várias máquinas que formam um *cluster*. Um *cluster* GFS consiste de vários nós, divididos em dois tipos: *master* e *chunk*.

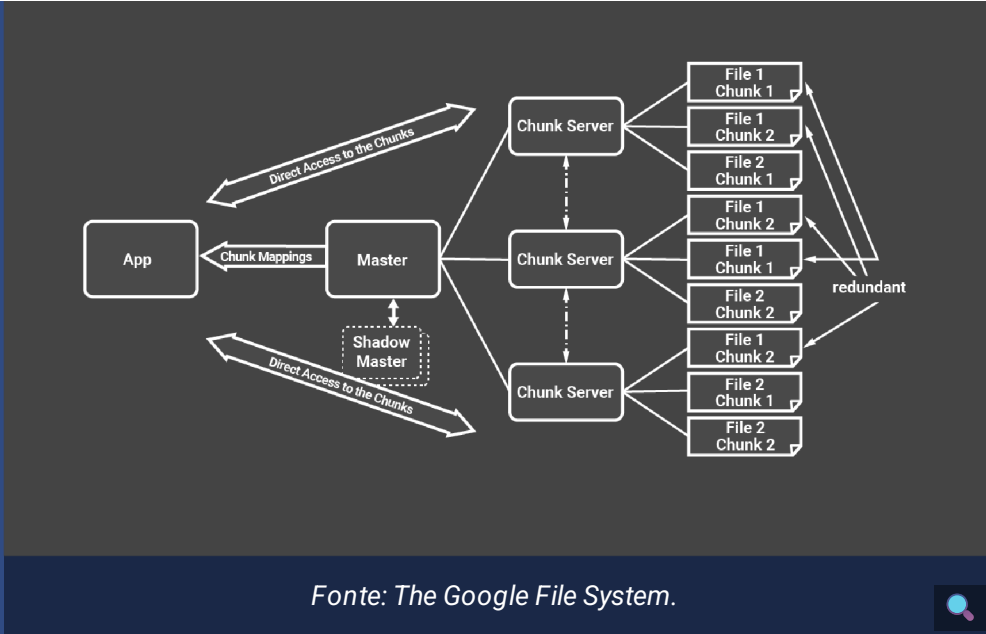
A próxima figura apresenta os principais componentes do GFS: uma aplicação cliente, um servidor GFS *master* e dois servidores GFS *chunk*.



A função do nó *master* é gerenciar os metadados do sistema de arquivos, mantendo informações de controle de acesso e o mapeamento de cada arquivo específico para o conjunto de blocos (*chunks*) associado. A aplicação cliente solicita essas informações de localização para o GFS *master*, e este devolve com as respostas (*chunk handle*, *chunk locations*).

A aplicação cliente solicita o *chunk handle* ao servidor *chunk*, e este, então, devolve com os dados associados ao *handle* e, alternativamente, a faixa de bytes solicitada. Note que o GFS *master* responde à primeira solicitação enviando também as *chunk locations*, porque cada *chunk* é replicado para *n* servidores, garantindo a redundância a falhas. Além disso, caso uma falha ocorra em um nó do *cluster*, o dado pode ser encontrado em outro nó. Assim, caso determinada *location* não esteja disponível, o cliente automaticamente seleciona a próxima *location* e realiza a solicitação. É esse comportamento que garante a tolerância a falhas no GFS. A configuração do número de réplicas pode ser feita no nível do arquivo, para cada *chunk* (ou seja, diferentes *chunks* de um mesmo arquivo podem ter réplicas em nós distintos).

Na representação a seguir, você pode visualizar que o bloco 1 do arquivo 1 está replicado no primeiro e no segundo servidores, mas o bloco 2 do arquivo 1 está replicado nos três servidores.



Um ponto-chave na arquitetura do GFS é o tamanho do *chunk* (64 Mbytes), bem maior que os tamanhos de bloco adotados em sistemas de arquivos tradicionais e bancos de dados. Isso permite que o GFS mantenha proporcionalmente menos metadados.

Se um tamanho de *chunk* de 64 KBytes fosse adotado, por exemplo, o volume de metadados aumentaria em um fator de 1.000. Isso, por sua vez, significa que os GFS *masters* geralmente podem manter todos os seus metadados na memória principal, diminuindo significativamente a latência para operações de controle. Por sua vez, note que o *master* está envolvido no início e depois fica completamente fora do *loop*, implementando uma separação de controle e fluxos de dados que é crucial para manter o alto desempenho dos acessos a arquivos.



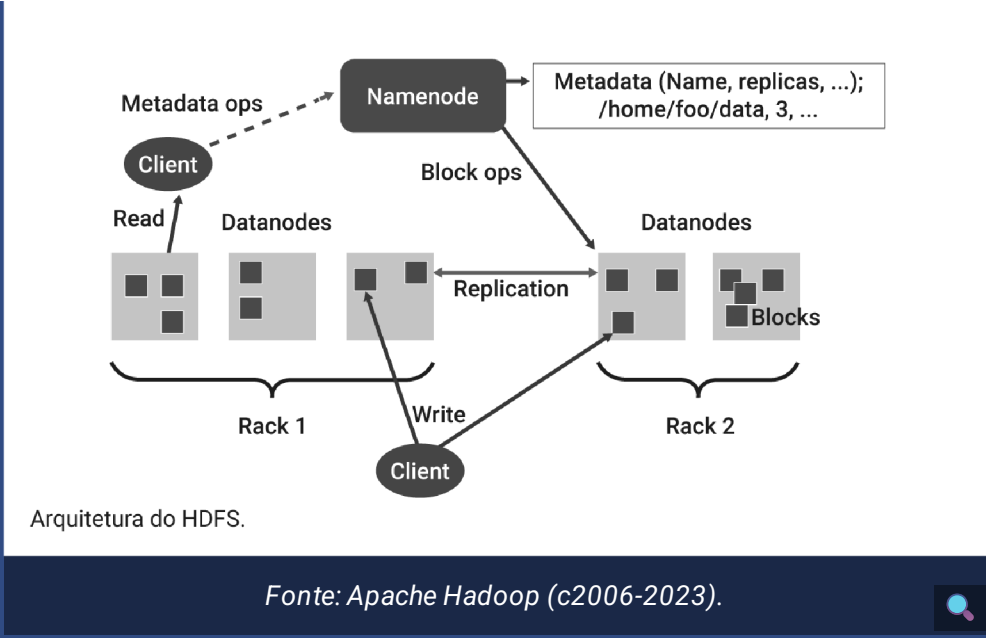
Fique Ligado

A Apache, que vinha enfrentando problemas de desempenho com o Nutch, passou a experimentar o GFS e o MapReduce e decidiu adotar os princípios estabelecidos pela Google e implementar algumas modificações no GFS, passando a chamá-lo de HDFS (Hadoop Distributed File System), criando o núcleo da **versão 1.0 do Hadoop**, composto pelo HDFS e pelo MapReduce.

O HDFS segue a abordagem *write-once-read-many* para seus arquivos e aplicativos. Ele assume que um arquivo em HDFS, uma vez gravado, não será modificado, embora possa ser acessado várias vezes. Versões futuras do Hadoop viriam a suportar esse recurso.

Note que um aplicativo MapReduce ou um *crawler* são bem adequados a essa aparente limitação do HDFS, que simplifica os problemas de coerência de dados. Como o HDFS é projetado mais para processamento em lote do que para uso interativo pelos usuários, a ênfase está no alto rendimento do acesso aos dados.

A figura a seguir ilustra a arquitetura do HDFS.



Na figura, *namenode* contém os metadados (lista de arquivos, lista de blocos para cada arquivo, lista de *datanodes* para cada bloco, atributos do arquivo, data de criação e fator de replicação) e um log de transações que registra as operações com cada arquivo (criação e exclusão).

A imagem do *namespace* (com os metadados do HDFS) se chama FSIMAGE. Durante a inicialização do *namenode*, o arquivo FSIMAGE é lido e carregado em memória. Quaisquer alterações em andamento nos arquivos e diretórios no FSIMAGE serão gravadas na memória e no arquivo de log de transações. *Namenode* não salva as alterações em andamento no FSIMAGE diretamente porque o arquivo FSIMAGE pode ser muito grande, e atualizar um arquivo como esse em tempo de execução pode ser muito caro.

É possível ainda configurar um *secondary namenode* que, apesar do nome, não atua como um *namenode*. Sua principal função é mesclar periodicamente a imagem do *namespace* com o arquivo de log de transações do sistema de arquivos, evitando que o log de edição fique muito grande.

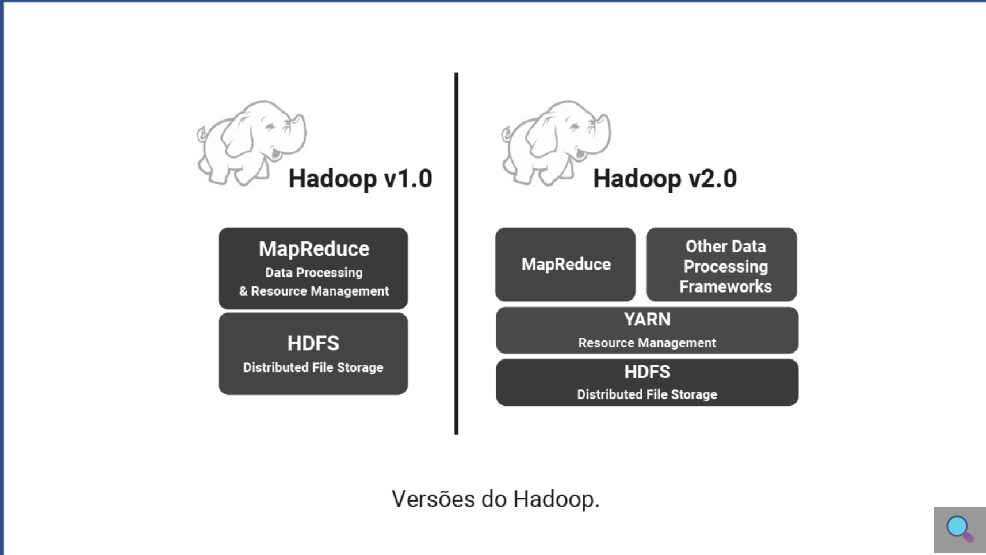
Hadoop 2.0 e a *stack* Hadoop Spark



Com o passar do tempo e as novas gerações de arquiteturas de BI (BI 2.0, 3.0, 4.0), a comunidade começou a sentir falta de *frameworks* para processamento interativo e em tempo real (já que o MapReduce trata originalmente apenas do processamento *batch*). O Hadoop 2.0, lançado pela Apache em 2013, visou a habilitar outros *frameworks* de processamento além do MapReduce, como o Yet Another Resource Negotiator (YARN).

O YARN foi criado para ser o gerenciador de recursos do *cluster*, operando sobre o HDFS. Com isso, o MapReduce passou a ser um *framework* para execução de processamento batch, coexistindo com outros *frameworks* que surgiram ao longo do tempo.

A figura a seguir ilustra a pilha (*stack*) Hadoop atualmente conhecida. Veja:



A principal diferença entre o YARN e o MapReduce é que o primeiro suporta uma variedade de motores (*engines*) de processamento de aplicações, enquanto o segundo suporta apenas o seu modelo de processamento *batch*. Além disso, o MapReduce é um componente único para realizar as tarefas que o YARN faz em componentes separados, como a alocação dinâmica de recursos do *cluster* para as aplicações. No MapReduce, essa alocação é estática. A ideia fundamental do YARN é dividir as responsabilidades dos componentes do MapReduce em entidades separadas.

Na arquitetura do Hadoop 2.0 há três componentes. Clique nos *cards* para conhecê-los.

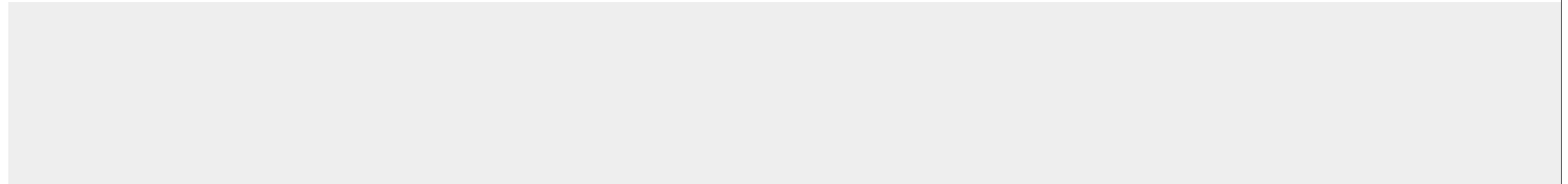
Interativo

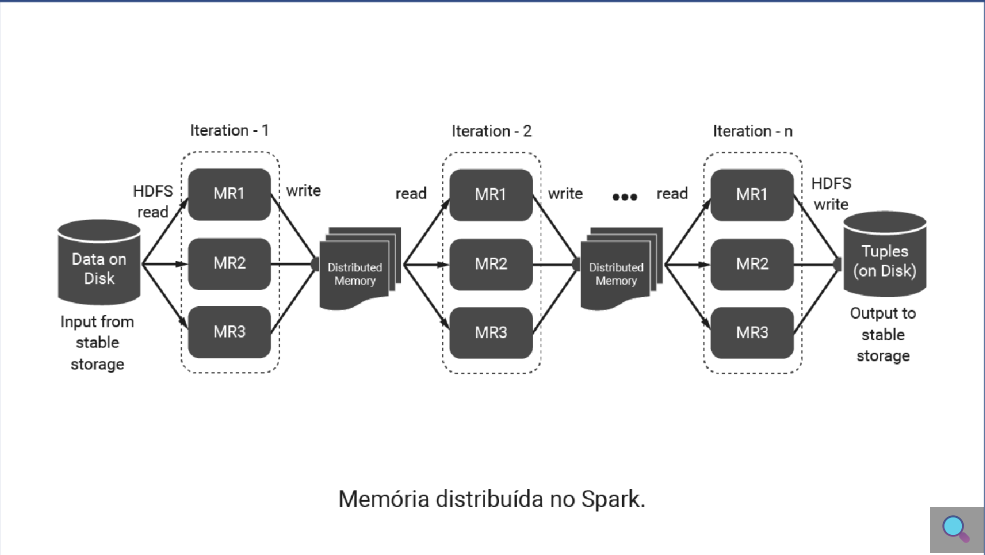
Descrição do interativo

Outra tecnologia importante da arquitetura Hadoop 2.0 é o Spark. O Spark foi criado para resolver um problema de degradação de desempenho no MapReduce, uma vez que a implementação de uma aplicação usando o modelo MapReduce pode exigir muitas tarefas Map e Reduce encadeadas, as quais geram muitos arquivos intermediários no disco, em tempo de execução. Com isso, o acesso aos dados se torna intensivo, aumentando demasiadamente o custo de operações complexas e degradando o desempenho da infraestrutura como um todo.

O Spark substitui estruturas em disco por estruturas em memória distribuída, eliminando o custo de I/O associado a múltiplas leituras e escritas em disco do MapReduce.

A figura a seguir ilustra a o processamento distribuído utilizando Spark. Analise-a.





O que antes era atualizado em arquivos em disco, agora, no Spark, é atualizado diretamente, em memória distribuída pelos nós do *cluster*. Dessa forma, dados intermediários do processamento são acessados pelas tarefas Map e Reduce diretamente na memória dos nós do *cluster*, acelerando o desempenho antes degradado dos *jobs* MapReduce.

Além da maior utilização da memória (em vez de disco) para execução de operações de processamento, uma das características principais do Spark é a execução de aplicações interativas, como, por exemplo, uma consulta SQL.

Dizemos que o Spark estende o MapReduce (que executa somente aplicações *batch*) na direção de aplicações interativas e em tempo real.

Visão da infraestrutura de dados de um *data lake* como ecossistema Hadoop Spark

Clusters Hadoop Spark são atualmente a base fundamental dos DL como plataformas de dados para BI, suportando tarefas de exploração de dados, integração de fontes heterogêneas e treinamento de modelos de inteligência artificial em larga escala. Por ser uma plataforma de dados apartada dos sistemas transacionais (OLTP), a ingestão dos dados que ocorre durante a Produção Batch não impacta o desempenho on-line dos sistemas corporativos.

Uma visão geral do ecossistema Hadoop Spark é ilustrada nesta figura:



Esse ecossistema é vivo e continua evoluindo a todo momento. O Ambari, por exemplo, foi desenvolvido inicialmente pela Hortonworks e é uma ferramenta para provisionamento, gerenciamento e monitoramento dos *clusters* Apache Hadoop. Já o Kafka é uma

plataforma *open source* de processamento de *streams* desenvolvida originalmente pelo LinkedIn (Confluent) e, depois, Apache. E o HBase é um banco de dados *open source* orientado a coluna e distribuído sobre o HDFS, modelado a partir do Google BigTable e escrito em Java.



Fique Ligado

Em nuvem, todos os fornecedores baseiam suas soluções em *clusters* Hadoop Spark, ainda que os componentes possam ser substituídos. É possível, por exemplo, utilizar o AWS S3 ou Azure Blob Storage em vez do HDFS para armazenamento distribuído, ou, ainda, utilizar o Mesos ou Kubernetes em vez do YARN para gerenciar os recursos distribuídos no *cluster*.

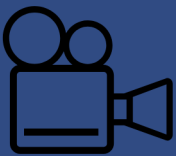
ETL e *data ingestion*

A ingestão de dados em um DL pode ser realizada de várias maneiras, dependendo das necessidades, dos sistemas envolvidos e dos recursos disponíveis. De fato, existem componentes no ecossistema Hadoop que podem realizar a transferência de dados para o DL, cada um com suas especificidades e aplicações mais adequadas. As abordagens e ferramentas mais comuns são:

Interativo

Interativo que contem 4 imagens

Diante dessas estratégias de ingestão de dados no DL, a escolha deverá levar em consideração aspectos característicos do tipo de dados que será ingerido, seu volume, vazão e destino. Um arquivo CSV destinado ao Hive pode ser copiado diretamente para o HDFS, mas dados oriundos de um serviço *streaming* em tempo real deverão passar por um consumidor adequado às taxas de transferência, capaz de receber os dados e persisti-los em um banco de dados adequado ao processamento posterior.



Infraestrutura de dados, ETL e *data lakes*



Nesta videoaula o professor Antony Seabra fala um pouco mais sobre infraestrutura de dados, ETL e *data lakes*, além disso, ele explica as atividades propostas no Técnica Aplicada para você poder praticar o que aprendeu nesta aula.



Referências

Clique aqui para acessar as referências e créditos desta aula.



ANDRADE, T. P. C. **MapReduce**: conceitos e aplicações. Disponível em: https://www.ic.unicamp.br/~cortes/mo601/trabalho_mo601/tiago. Acesso em: 12 jun. 2023.

APACHE SPARK RDD. **Tutorials point**. Disponível em: https://www.tutorialspoint.com/apache_spark/apache_spark_rdd.h. Acesso em: 12 jun. 2023.

APACHE HADOOP. c2006-2023. Disponível em: <https://hadoop.apache.org/>. Acesso em: 15 jun. 2023.

BHANDARKAR, M. AdBench: a complete benchmark for modern data pipelines. In: NAMBIAR, R.; POESS, M. (Eds.). **Performance evaluation and benchmarking. traditional - big data - internet of things**. TPCTC 2016. Lecture Notes in Computer Science. Cham: Springer. p. 107-120.

COUTO, J. A **Model for automatized data integration in Hadoop-based data lakes**. Tese. (Doutorado em Ciência da Computação) – Pontifícia Universidade Católica do Rio Grande do Sul, Porto Alegre, 2022.

DEAN, J.; GHEMAWAT, S. MapReduce: simplified data processing on large clusters. **Communications of the ACM**, v. 51, n. 1, 2004.

GHEMAWAT, S.; GOBIOFF, H.; LEUNG, S. T. The Google File System. In: THE NINETEENTH ACM SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES, 2003, New York. **Proceedings** [...]. The Sagamore, Bolton Landing (Lake George), New York, 2003. p. 29-43. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/945445.945450> Acesso em: 15 jun. 2003.

GRÖGER, C. Building an industry 4.0 analytics platform. **Datenbank-Spektrum**, v. 18, p. 5-14, 2018.

KATHIRAVELU, P.; SHARMA, A. A dynamic data warehousing platform for creating and accessing biomedical data lakes. In: WANG, F., YAO, L., LUO, G. (Eds.). **Data management and**

analytics for medicine and healthcare. DMAH 2016. Lecture Notes in Computer Science. Cham: Springer. p. 101-120.

MCPADDEN, J. *et al*/8888888. A scalable data science platform for healthcare and precision medicine research. **Computing Research Repository - arXiv**, v. abs/1808.04849, p. 1–7, 2018.

NOGUEIRA, I. D.; ROMDHANE, M.; DARMONT, J. Modeling data lake metadata with a data vault. **Computing Research Repository - arXiv**, v. abs/1807.04035, p. 1–19, 2018.

TAHER, Y. *et al*. A context-aware analytics for processing tweets and analysing sentiment in realtime. In: CONFEDERATED INTERNATIONAL CONFERENCES: ON THE MOVE TO MEANINGFUL INTERNET SYSTEMS, p. 910–917. Rhodes, GR: Springer, 2016.

WHAT IS MapReduce? **IntelliPaat**, 2023. Disponível em: <https://intellipaate.com/blog/what-is-mapreduce/>. Acesso em: 15 jun. 2023.

Banco de imagens Envato, Freepik, Pexels, Pixabay e Shutterstock.

